

특정 인물 식별 코드 정리

YOLO + MediaPipe Tasks + InsightFace 기반 실시간 얼굴 식별 코드

이 문서는 웹캠 또는 RTSP 영상에서 객체를 탐지하고, 사람 객체에 대해서만 얼굴을 검출한 뒤, 등록된 얼굴 데이터와 비교하여 특정 인물을 식별하는 코드를 정리한 문서이다. 전체 코드를 먼저 제시하고, 이후 각 함수와 흐름이 어떤 역할을 하는지 순서대로 설명한다.

1. 전체 코드

```
import cv2
import numpy as np
import mediapipe as mp
from pathlib import Path
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
from ultralytics import YOLO
from insightface.app import FaceAnalysis

RTSP_URL = "rtsp://192.168.10.2:8554/stream"
VIDEO_SOURCE = 0

YOLO_MODEL_PATH = "yolo26s.pt"
FACE_TASK_MODEL_PATH = "blaze_face_short_range.tflite"
KNOWN_FACE_DIR = "known_faces"

TARGET_CLASSES = ["person", "apple"]
YOLO_CONF_TH = 0.5
FACE_SIM_THRESHOLD = 0.20

yolo_model = YOLO(YOLO_MODEL_PATH)

face_app = FaceAnalysis(name="buffalo_l", providers=["CPUExecutionProvider"])
face_app.prepare(ctx_id=-1, det_size=(640, 640))

def normalize_embedding(embedding: np.ndarray) -> np.ndarray:
    emb = embedding.astype(np.float32)
    norm = np.linalg.norm(emb) + 1e-8
    return emb / norm

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b))

def get_largest_face(faces):
    if not faces:
        return None
    return max(
        faces,
        key=lambda f: (f.bbox[2] - f.bbox[0]) * (f.bbox[3] - f.bbox[1])
    )

def expand_box(x1, y1, x2, y2, img_w, img_h, scale=0.4):
```

```
w = x2 - x1
h = y2 - y1
dx = int(w * scale)
dy = int(h * scale)
```

```
nx1 = max(0, x1 - dx)
ny1 = max(0, y1 - dy)
nx2 = min(img_w, x2 + dx)
ny2 = min(img_h, y2 + dy)
return nx1, ny1, nx2, ny2
```

```
def load_known_faces(face_dir: str):
```

```
    face_db = {}
    base = Path(face_dir)
```

```
    if not base.exists():
        print(f"[경고] 등록 얼굴 폴더가 없습니다: {face_dir}")
        return face_db
```

```
    for person_dir in base.iterdir():
        if not person_dir.is_dir():
            continue
```

```
    person_name = person_dir.name
    embeddings = []
```

```
    for img_path in person_dir.iterdir():
        if img_path.suffix.lower() not in [".jpg", ".jpeg", ".png", ".bmp"]:
            continue
```

```
    img = cv2.imread(str(img_path))
    if img is None:
        continue
```

```
    faces = face_app.get(img)
    if not faces:
        print(f"[등록 실패] {person_name}: 얼굴 검출 안됨 - {img_path.name}")
        continue
```

```
    face = get_largest_face(faces)
    emb = normalize_embedding(face.embedding)
    embeddings.append(emb)
```

```
    if embeddings:
        face_db[person_name] = embeddings
        print(f"[등록 완료] {person_name}: {len(embeddings)}장")
    else:
        print(f"[등록 실패] {person_name}: 사용 가능한 얼굴 이미지 없음")
```

```
    return face_db
```

```
def recognize_face(face_crop: np.ndarray, face_db: dict, threshold=0.20):
```

```
    faces = face_app.get(face_crop)
    if not faces:
        return "Unknown", -1.0
```

```
    face = get_largest_face(faces)
    emb = normalize_embedding(face.embedding)
```

```

best_name = "Unknown"
best_score = -1.0

for person_name, emb_list in face_db.items():
    scores = [cosine_similarity(emb, ref_emb) for ref_emb in emb_list]
    score = max(scores)

    if score > best_score:
        best_score = score
        best_name = person_name

if best_score < threshold:
    return "Unknown", best_score

return best_name, best_score

def detect_face_with_tasks(face_detector, bgr_roi: np.ndarray):
    if bgr_roi is None or bgr_roi.size == 0:
        return None

    rgb_roi = cv2.cvtColor(bgr_roi, cv2.COLOR_BGR2RGB)
    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_roi)
    result = face_detector.detect(mp_image)

    if not result.detections:
        return None

    h, w, _ = bgr_roi.shape
    best_box = None
    best_area = -1

    for detection in result.detections:
        bbox = detection.bounding_box
        x1 = int(bbox.origin_x)
        y1 = int(bbox.origin_y)
        x2 = int(bbox.origin_x + bbox.width)
        y2 = int(bbox.origin_y + bbox.height)

        x1 = max(0, x1)
        y1 = max(0, y1)
        x2 = min(w, x2)
        y2 = min(h, y2)

        area = max(0, x2 - x1) * max(0, y2 - y1)
        if area > best_area:
            best_area = area
            best_box = (x1, y1, x2, y2)

    return best_box

face_db = load_known_faces(KNOWN_FACE_DIR)

if not face_db:
    print("[경고] 등록 얼굴 DB 가 비어 있습니다. Unknown 만 표시될 수 있습니다.")

base_options = python.BaseOptions(model_asset_path=FACE_TASK_MODEL_PATH)

```

```

options = vision.FaceDetectorOptions(
    base_options=base_options,
    running_mode=vision.RunningMode.IMAGE,
    min_detection_confidence=0.5,
    min_suppression_threshold=0.3,
)

cap = cv2.VideoCapture(VIDEO_SOURCE)

if not cap.isOpened():
    print("영상 연결 실패")
    exit()

print("영상 연결 성공")

with vision.FaceDetector.create_from_options(options) as face_detector:
    while True:
        ret, frame = cap.read()

        if not ret:
            print("프레임 수신 실패")
            continue

        results = yolo_model(frame, verbose=False)
        result = results[0]

        for box in result.bboxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            class_name = yolo_model.names[cls_id]

            if class_name not in TARGET_CLASSES:
                continue
            if conf < YOLO_CONF_TH:
                continue

            x1, y1, x2, y2 = map(int, box.xyxy[0])

            x1 = max(0, x1)
            y1 = max(0, y1)
            x2 = min(frame.shape[1], x2)
            y2 = min(frame.shape[0], y2)

            box_color = (0, 255, 0)
            label = f'{class_name} {conf:.2f}'

            if class_name == "person":
                person_roi = frame[y1:y2, x1:x2]
                face_box = detect_face_with_tasks(face_detector, person_roi)

                if face_box is not None:
                    fx1, fy1, fx2, fy2 = face_box

                    fx1_global = x1 + fx1
                    fy1_global = y1 + fy1
                    fx2_global = x1 + fx2
                    fy2_global = y1 + fy2

```

```

ex1, ey1, ex2, ey2 = expand_box(
    fx1_global, fy1_global, fx2_global, fy2_global,
    frame.shape[1], frame.shape[0],
    scale=0.4
)

face_crop = frame[ey1:ey2, ex1:ex2]

if face_crop.size > 0:
    person_name, sim = recognize_face(
        face_crop,
        face_db,
        threshold=FACE_SIM_THRESHOLD
    )

    if person_name != "Unknown":
        label = f"{person_name} / person {conf:.2f} / sim {sim:.2f}"
        box_color = (255, 0, 0)
    else:
        label = f"Unknown / person {conf:.2f} / sim {sim:.2f}"
        box_color = (0, 165, 255)

    cv2.rectangle(
        frame,
        (fx1_global, fy1_global),
        (fx2_global, fy2_global),
        (255, 255, 0),
        2
    )
else:
    label = f"person {conf:.2f} / face crop fail"
else:
    label = f"person {conf:.2f} / face not found"

cv2.rectangle(frame, (x1, y1), (x2, y2), box_color, 2)
cv2.putText(
    frame,
    label,
    (x1, max(20, y1 - 10)),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.55,
    box_color,
    2
)

cv2.imshow("YOLO + MediaPipe Tasks(IMAGE) + InsightFace", frame)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

```

```

cap.release()
cv2.destroyAllWindows()

```

2. 코드 역할 설명

import 부분

OpenCV, NumPy, MediaPipe Tasks, YOLO, InsightFace 를 불러오는 부분이다. 영상 입력, 객체 탐지, 얼굴 검출, 얼굴 특징 비교에 필요한 모듈만 사용한다.

기본 설정

RTSP 주소, 입력 소스, 모델 파일 경로, 등록 얼굴 폴더 경로, 탐지 대상 클래스, YOLO confidence 기준값, 얼굴 비교 기준값을 한 곳에 모아 둔 부분이다. 테스트 단계에서는 VIDEO_SOURCE 를 0 으로 두면 웹캠, RTSP_URL 로 두면 라즈베리파이 스트림으로 실행할 수 있다.

YOLO 모델 로드

YOLO 는 프레임 전체에서 person, apple 같은 객체를 찾는 역할을 한다. 이 코드에서는 먼저 YOLO 가 사람을 찾고, 그 사람 박스 안에서만 얼굴 검출을 진행하도록 구성했다.

InsightFace 로드

InsightFace 는 얼굴 이미지를 숫자 벡터 형태의 임베딩으로 바꾸는 역할을 한다. 등록된 얼굴 사진과 현재 잘라낸 얼굴을 같은 방식으로 임베딩으로 바꾼 뒤, 유사도를 비교해 가장 비슷한 사람을 결정한다.

normalize_embedding()

얼굴 임베딩 벡터를 정규화하는 함수다. 비교 전에 벡터 길이를 맞춰 두면 코사인 유사도 계산이 더 안정적이다.

cosine_similarity()

현재 얼굴 임베딩과 등록 얼굴 임베딩 사이의 유사도를 구한다. 값이 높을수록 같은 사람일 가능성이 크다.

get_largest_face()

얼굴이 여러 개 검출되면 가장 큰 얼굴 하나만 선택한다. 등록 사진과 사람 ROI 모두 이 기준으로 처리한다.

expand_box()

검출된 얼굴 박스를 조금 넓혀서 crop 범위를 만든다. 얼굴만 너무 딱 맞게 자르면 인식이 흔들릴 수 있어서 여유를 주는 역할이다.

load_known_faces()

known_faces 폴더를 읽어서 사람별 얼굴 임베딩 데이터베이스를 만든다. 예를 들어 known_faces/father 폴더에 사진을 여러 장 넣어 두면, 각 사진에서 얼굴을 찾아 임베딩을 추출하고 father 라는 이름으로 저장한다.

recognize_face()

현재 잘라낸 얼굴을 등록 얼굴 DB 와 비교하는 핵심 함수다. 각 사람에 대해 여러 장의 등록 사진이 있을 수 있으므로, 그중 가장 높은 유사도 점수를 사용한다. 가장 높은 점수도 threshold 보다 낮으면 Unknown 으로 처리한다.

detect_face_with_tasks()

MediaPipe Tasks API 로 사람 ROI 안에서 얼굴을 찾는 함수다. 여기서는 IMAGE 모드를 사용하므로 detect() 를 호출한다. 같은 프레임에서 사람 ROI 가 여러 개 나올 수 있기 때문에 VIDEO 모드보다 IMAGE 모드가 더 자연스럽다.

등록 얼굴 DB 로드

실행 시작 전에 등록 얼굴 데이터베이스를 메모리에 올린다. 등록 폴더가 비어 있으면 Unknown 만 출력될 수 있으므로 경고 메시지를 남긴다.

MediaPipe Face Detector 설정

blaze_face_short_range.tflite 모델 파일 경로를 BaseOptions 에 넣고 FaceDetectorOptions 를 생성한다. running_mode 는 IMAGE 로 두어 ROI 단위 얼굴 검출에 맞췄다.

영상 입력 열기

cv2.VideoCapture 로 입력 스트림을 연다. 웹캠과 RTSP 중 어떤 소스를 사용할지는 VIDEO_SOURCE 값으로 결정된다.

메인 루프 전체 흐름

프레임을 한 장씩 읽고, 먼저 YOLO 로 객체를 탐지한다. 탐지된 객체 중 TARGET_CLASSES 에 포함되고 confidence 기준을 넘는 것만 사용한다. person 일 때는 그 사람 박스 내부로 ROI 로 잘라 MediaPipe 로 얼굴을 찾고, 얼굴이 검출되면 얼굴 영역을 조금 넓혀 crop 한 뒤 InsightFace 로 등록 얼굴 DB 와 비교한다. 최종적으로 객체 박스, 얼굴 박스, 이름 또는 Unknown 라벨을 화면에 표시한다.

3. 폴더 구조 예시

```
project/
├─ main.py
├─ yolo26s.pt
├─ blaze_face_short_range.tflite
└─ known_faces/
    ├─ father/
    │   ├─ 1.jpg
    │   └─ 2.jpg
    ├─ mother/
    │   ├─ 1.jpg
    │   └─ 2.jpg
    └─ brother/
        ├─ 1.jpg
        └─ 2.jpg
```

등록 얼굴 폴더에는 사람별로 정면 또는 약간 각도가 다른 사진을 여러 장 넣어 두는 것이 좋다. 사진이 너무 적거나 얼굴이 작게 보이면 Unknown 비율이 높아질 수 있다.

4. 실행 흐름 요약

- 1) 입력 영상 한 프레임을 읽는다.
- 2) YOLO 로 객체를 탐지한다.
- 3) person 객체만 따로 얼굴 검출 단계로 보낸다.
- 4) MediaPipe Tasks API 로 사람 ROI 안의 얼굴을 검출한다.
- 5) 검출된 얼굴을 crop 한 뒤 InsightFace 임베딩을 계산한다.
- 6) 등록 얼굴 DB 와 비교해 가장 비슷한 사람을 찾는다.
- 7) 기준값보다 낮으면 Unknown, 넘으면 해당 사람 이름을 출력한다.
- 8) 최종 박스와 라벨을 프레임 위에 그린다.

5. 출력 예시

- father / person 0.91 / sim 0.82
- Unknown / person 0.88 / sim 0.14
- apple 0.77

정리하면 이 코드는 YOLO 가 객체 위치를 찾고, MediaPipe Tasks 가 얼굴 위치를 찾고, InsightFace 가 등록된 사람과의 유사도를 비교하는 구조라고 보면 된다. 즉 객체 탐지, 얼굴 검출, 얼굴 식별을 단계별로 분리해서 처리하는 코드다.